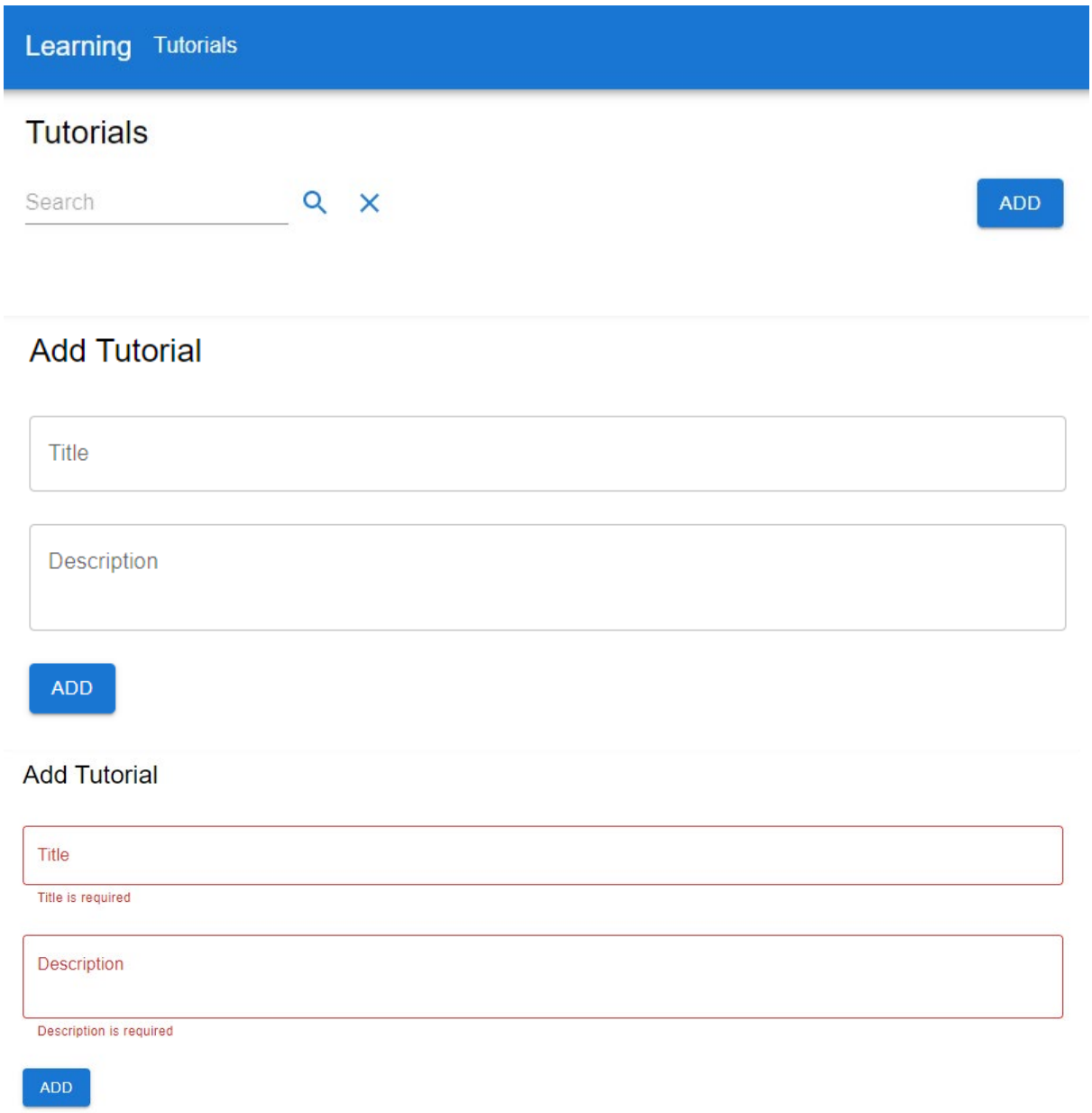
Lab4a Search and Add Data

Learning Outcomes

- Add function to search tutorial
- Add new page to add tutorial
- Validate input and submit form



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 1: Add function to search tutorial

1. Use VS Code to open the folder **learning-app**.

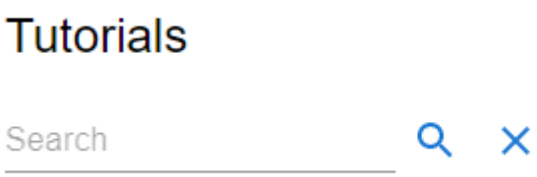
2. Open 2 terminals, one for **server**, the other for **client**. Start the server and the client.

3. Under the client folder, open src/pages/Tutorials.jsx, add the state and components for search:

```
import { Box, Typography, Grid, Card, CardContent, Input, IconButton } from
'@mui/material';
import { AccessTime, Search, Clear } from '@mui/icons-material';
…
    const [tutorialList, setTutorialList] = useState([]);
    const [search, setSearch] = useState('');

    const onSearchChange = (e) => {
        setSearch(e.target.value);
    };
…
            <Typography variant="h5" sx={{ my: 2 }}>
                Tutorials
            </Typography>

            <Box sx={{ display: 'flex', alignItems: 'center', mb: 2 }}>
                <Input value={search} placeholder="Search"
                    onChange={onSearchChange} />
                <IconButton color="primary">
                    <Search />
                </IconButton>
                <IconButton color="primary">
                    <Clear />
                </IconButton>
            </Box>
```

The input value uses the state **search**. The input on change event handler **onSearchChange** sets the state based on the input value.
The buttons are to search tutorials and clear search.

4. In the browser, observe the search input and buttons are displayed.



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. Go to VS Code, in Tutorials.jsx, edit the code to implement search and clear functions:

```
    const onSearchChange = (e) => {
        setSearch(e.target.value);
    };

    const getTutorials = () => {
        http.get('/tutorial').then((res) => {
            setTutorialList(res.data);
        });
    };

    const searchTutorials = () => {
        http.get(`/tutorial?search=${search}`).then((res) => {
            setTutorialList(res.data);
        });
    };

    useEffect(() => {
        getTutorials();
    }, []);

    const onSearchKeyDown = (e) => {
        if (e.key === "Enter") {
            searchTutorials();
        }
    };

    const onClickSearch = () => {
        searchTutorials();
    }

    const onClickClear = () => {
        setSearch('');
        getTutorials();
    };
```

Function getTutorials: call api to get all tutorials
Function searchTutorials: call api to search tutorials
Function useEffect: call function getTutorials
Function onSeachKeyDown: when enter key is down, call function searchTutorials
Function onClickSearch: call function searchTutorials
Function onClickClear: clear search state and call function getTutorials

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

6. Add the event handlers for the components:

```
            <Box sx={{ display: 'flex', alignItems: 'center', mb: 2 }}>
                <Input value={search} placeholder="Search"
                    onChange={onSearchChange}
                    onKeyDown={onSearchKeyDown} />
                <IconButton color="primary"
                    onClick={onClickSearch}>
                    <Search />
                </IconButton>
                <IconButton color="primary"
                    onClick={onClickClear}>
                    <Clear />
                </IconButton>
            </Box>
```

7. In the browser, test the search function. You can search by title or description.

Activity 2: Add new page to add tutorial

1. In VS Code, under the folder client/src/pages, add a new file AddTutorial.jsx

2. In AddTutorial.jsx, type **rfce** to generate the starting code:

```
import React from 'react'

function AddTutorial() {
  return (
    <div>AddTutorial</div>
  )
}

export default AddTutorial
```

3. In App.jsx, add a new route to add tutorial:

```
import AddTutorial from './pages/AddTutorial';
…

          <Route path={"/tutorials"} element={<Tutorials />} />
          <Route path={"/addtutorial"} element={<AddTutorial />} />
```


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

4. In Tutorials.jsx, add link button to add tutorial:

```
import { Link } from 'react-router-dom';
import { Box, Typography, Grid, Card, CardContent, Input, IconButton, Button }
from '@mui/material';
…
                <IconButton color="primary"
                    onClick={onClickClear}>
                    <Clear />
                </IconButton>
                <Box sx={{ flexGrow: 1 }} />
                <Link to="/addtutorial" style={{ textDecoration: 'none' }}>
                    <Button variant='contained'>
                        Add
                    </Button>
                </Link>
```

5. In the browser, click the button Add to go to the add tutorial page.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

6. In AddTutorial.jsx, change code to display page title and form:

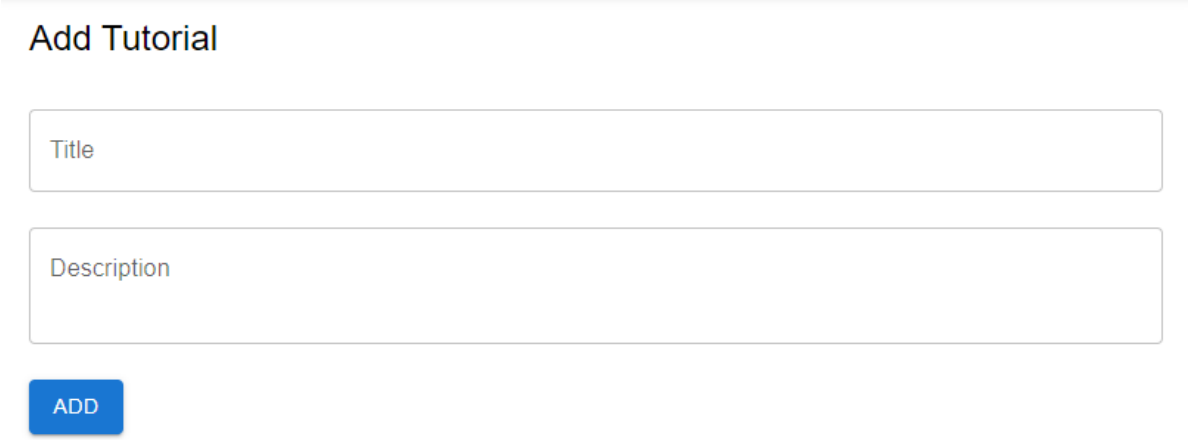
```
import React from 'react';
import { Box, Typography, TextField, Button } from '@mui/material';

function AddTutorial() {
    return (
        <Box>
            <Typography variant="h5" sx={{ my: 2 }}>
                Add Tutorial
            </Typography>
            <Box component="form">
                <TextField
                    fullWidth margin="dense" autoComplete="off"
                    label="Title"
                    name="title"
                />
                <TextField
                    fullWidth margin="dense" autoComplete="off"
                    multiline minRows={2}
                    label="Description"
                    name="description"
                />
                <Box sx={{ mt: 2 }}>
                    <Button variant="contained" type="submit">
                        Add
                    </Button>
                </Box>
            </Box>
        </Box>
    );
}

export default AddTutorial;
```

For the component Box, you can use component="form" to indicate the html element for the Box is form.

Inside the form, there are 2 text fields for title and description, like html input. The text filed description uses multiline.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

7. In the browser, observe that the form is displayed below the page title.



8. Stop the client app.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 3: Validate input and submit form

1. In the client terminal, run command to install package formik and yup:
   npm i formik yup
   Formik is a popular form library for React.
   Yup is a JavaScript schema builder for value parsing and validation.

2. In AddTutorial.jsx, add code to use formik and yup:

```
import { useFormik } from 'formik';
import * as yup from 'yup';
import http from '../http';

function AddTutorial() {
    const formik = useFormik({
        initialValues: {
            title: "",
            description: ""
        },
        validationSchema: yup.object({
            // code to be added…
        }),
        onSubmit: (data) => {
            data.title = data.title.trim();
            data.description = data.description.trim();
            http.post("/tutorial", data)
                .then((res) => {
                    console.log(res.data);
                });
        }
    });
    …
}

export default AddTutorial;
```

The function **useFormik** is to create the formik instance with initial values, validation schema and on submit function.

Yup is used to define the validation schema, similar to what we did in the server post request.

When the form is submitted, sanitize the string inputs and call the api post function to add a tutorial.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

3. Add validations for input title and description:

```
validationSchema: yup.object({
    title: yup.string().trim()
        .min(3, 'Title must be at least 3 characters')
        .max(100, 'Title must be at most 100 characters')
        .required('Title is required'),
    description: yup.string().trim()
        .min(3, 'Description must be at least 3 characters')
        .max(500, 'Description must be at most 500 characters')
        .required('Description is required')
}),
```

Custom messages are added for all the validations.

4. Use formik for the title text field:

```
<TextField
    fullWidth margin="normal" autoComplete="off"
    label="Title"
    name="title"
    value={formik.values.title}
    onChange={formik.handleChange}
    onBlur={formik.handleBlur}
    error={formik.touched.title && Boolean(formik.errors.title)}
    helperText={formik.touched.title && formik.errors.title}
/>
```

5. Try yourself to apply the same for the description text field.

6. Add the on submit handler for the form:

```
<Box component="form" onSubmit={formik.handleSubmit}>
```

7. Start the client. In the browser, test the input validations.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

8. Enter title and description, click on the button Add. Observe that the new tutorial object is logged in the browser console.

9. In the app bar, click the link Tutorials. Observe that the new tutorial is displayed in the list.

10. Go to VS Code, in AddTutorial.jsx, add navigation after getting api response:

```
import { useNavigate } from 'react-router-dom';
…
function AddTutorial() {
    const navigate = useNavigate();
…
            http.post("/tutorial", data)
                .then((res) => {
                    console.log(res.data);
                    navigate("/tutorials");
                });
```

The useNavigate hook returns a function that lets you navigate programmatically.

11. In the browser, add a new tutorial. Observe that now the page is navigated to the tutorials page now.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.